

Ledger

Product Requirements Document

SMS-Native Personal Finance Agent for Informal & Multi-Bank Markets

Version 1.0 | Owner: Solo Founder/Engineer | Target Build Window: 2 Weeks

1. Problem Statement

People who earn and spend through mobile money and bank transfers (CBE, Telebirr, Awash, Dashen, and similar) routinely lose track of where their money goes within days of being paid. Existing budgeting apps solve this for users in banking systems they support, and they all share the same failure mode everywhere else: they require the user to manually log every purchase, and manual logging is abandoned within days by almost everyone who tries it. Meanwhile, the transaction record already exists — every debit or credit already arrives as an SMS notification the user reads once and ignores.

Ledger's core bet: if the app reads the SMS that already exists instead of asking the user to type anything, the effort barrier that kills every other budgeting app disappears.

2. Goals & Success Metrics

Goal	Metric	Target
User understands their spending without manual entry	% of transactions auto-categorized with no user input needed	≥ 80%
Weekly report is genuinely useful, not just a chart dump	% of test users who read the narrative summary (not just the chart)	Qualitative, tracked via 5–10 user interviews
System handles real multi-bank format diversity	Number of distinct bank/telecom SMS formats supported at launch	≥ 3 (CBE, Telebirr, one more)
Demonstrates senior backend architecture	Clean parser-plugin extensibility; documented in README/demo	Qualitative — portfolio review

3. Target Users

3.1 Primary Persona — "Abel, the salaried/gig earner"

Uses CBE and Telebirr daily for salary, transfers, and airtime. Gets paid monthly or per-gig, and by the second week doesn't know where a third of it went. Owns an Android phone. Has tried budgeting apps before and abandoned them within a week because logging expenses felt like a chore on top of an already busy day.

3.2 Secondary Persona — "Selam, the iOS or statement-only user"

Same underlying need, but on a platform where SMS capture isn't possible. Willing to upload a monthly bank statement instead, as a lower-frequency alternative.

4. Scope

4.1 In Scope (MVP)

- Android client: SMS capture, local sender-ID filtering, secure forwarding to backend.
- Go backend: ingestion API, parsers for at least 2–3 real bank/telecom SMS formats, categorization with a learning loop, agent orchestrator for clarifying questions and anomaly flags, scheduled weekly report generation.
- Web dashboard: login, balance/category view, transaction list, pending clarifying questions, weekly report view.
- Statement (PDF/CSV) upload as a universal fallback ingestion path.

4.2 Out of Scope (MVP)

- Play Store distribution (direct APK / sideload only for v1 — see PRD Section 7).
- iOS SMS capture (not possible on the platform; statement upload is the iOS path).
- On-device/offline LLM inference for the narrative summary (server-side LLM call only for v1).
- Bill negotiation, remittance advice, or any action beyond visibility and reporting.

5. User Stories

ID	As a...	I want to...	So that...
US-1	user	grant SMS permission once during onboarding	I never have to manually enter a transaction again
US-2	user	see my spending broken into categories automatically	I understand where my money goes without effort
US-3	user	answer a quick clarifying question when the app is unsure	categorization gets more accurate over time
US-4	user	receive a short written weekly summary, not just a chart	I understand *why* my spending changed, not just that it did
US-5	user	upload a bank statement instead of using SMS capture	I can use the app even on a phone where SMS capture isn't possible
US-6	user	be told clearly why the app needs SMS access before I grant it	I trust the app with my financial data

6. Core User Flow

6.1 Onboarding

- User downloads the APK directly (sideload) or opens the web dashboard.
- User registers an account on the dashboard.
- User installs the Android client, is shown a plain-language explanation of SMS access, and grants permission.
- Client shows a short list of detected historical bank/telecom messages found on-device as a first-run preview (read-only), confirming the app is working.

6.2 Ongoing Use (Passive)

- New bank/telecom SMS arrives → client filters → forwards → backend parses, categorizes, stores.
- If confidence is low, a clarifying question appears on the dashboard for the user to answer whenever convenient — not a blocking interruption.
- Once a week, the report worker runs and the user is notified their weekly report is ready.

6.3 Weekly Report

- User opens the dashboard or email link.
- Sees a category breakdown chart, a week-over-week comparison, and a short narrative explanation of the biggest driver(s) of change.

7. Distribution Strategy

Given the target market's actual app-acquisition behavior, Ledger ships v1 as a directly downloadable, signed APK (sideload) rather than through the Google Play Store, avoiding developer account cost and review latency. The landing page hosting the APK link will include a short, clear walkthrough of the Android "install from unknown sources" step, since this is the main adoption friction point for this distribution method.

8. Risks & Mitigations

Risk	Mitigation
Users distrust an app reading their SMS.	Local sender-ID filtering (non-financial SMS never leaves the device); explicit, upfront explanation before the permission prompt; in-app capture log for transparency.
Bank/telecom SMS format changes silently and breaks parsing.	Unmatched messages route to a review queue instead of failing silently; parser-plugin architecture makes fixes isolated and fast.
Sideloaded APK triggers Android security warnings and scares users off.	Landing page explains the warning proactively; long-term path to Play Store noted as future work.

Risk	Mitigation
LLM-generated narrative says something inaccurate or irrelevant.	Narrative generation is grounded strictly in the user's own structured transaction data for the period, not open-ended generation.

9. Milestones (2-Week Build Sprint)

Days	Deliverable
1–2	Go backend skeleton, data model, ingestion API (SMS + statement upload endpoints).
3–5	Parser modules for target bank/telecom formats; categorization engine with correction storage.
6–7	Android client: permission flow, broadcast receiver, filtering, forwarding, offline queue.
8–9	Agent orchestrator: clarifying-question logic, anomaly detection, LLM-backed narrative generation.
10–11	Scheduled weekly report worker; web dashboard views (balances, transactions, report, clarifications).
12–13	End-to-end testing with real captured SMS samples; APK signing and landing page.
14	Documentation, demo recording, LinkedIn/portfolio write-up.

10. Open Questions

- Which 2–3 bank/telecom formats to prioritize first for the parser layer (CBE + Telebirr are the strongest initial candidates given user reach).
- Email delivery of the weekly report vs. dashboard-only for v1, given added infrastructure (transactional email provider).
- Whether the review queue for unmatched SMS formats should be user-visible or engineer-only during the sprint.